

Comprendre le code du projet BraTS 2026 — Task 3 (GoAT)

Pour qui ? Ce document s'adresse à une personne qui ne connaît *rien* à l'intelligence artificielle appliquée à l'image médicale. Chaque terme technique est expliqué en français courant avant d'être utilisé. À la fin, vous comprendrez à quoi sert *chaque fichier de code* du projet et comment ils s'enchaînent.

Note d'honnêteté importante. Tout le code décrit ici existe et est testé (172 tests automatiques passent, Phase 4 SSL incluse), **mais aucun entraînement réel du modèle n'a encore été lancé**. Le projet est une « usine » complète et vérifiée ; les données sont désormais converties et le GPU est prêt — il manque le run d'entraînement lui-même et le réglage final des boutons.

Sommaire

1. Le problème en une page
 2. Le vocabulaire indispensable (expliqué simplement)
 3. Vue d'ensemble : le schéma du pipeline
 4. Les 8 phases du projet, fichier par fichier, fonction par fonction
 5. Les deux idées les plus astucieuses du code
 6. État actuel et ce qu'il reste à faire
 7. Mini-glossaire
-

1. Le problème en une page

Un **IRM cérébral** est une photo en trois dimensions du cerveau (un empilement de coupes, comme un pain de mie tranché). Quand un patient a une tumeur, on aimerait que l'ordinateur **colorie automatiquement** la tumeur sur l'image : c'est ce qu'on appelle la **segmentation**. Le médecin obtient ainsi, sans tout dessiner à la main, le volume et la forme exacts de la tumeur.

Le concours **BraTS 2026** organise une compétition mondiale sur ce sujet. La **Task 3**, surnommée **GoAT** (*Generalizability Across Tumors*, « généralisation à travers les tumeurs »), pose un défi précis :

Construire **un seul** programme de segmentation qui fonctionne bien sur **cinq types de tumeurs différentes** et sur des images venant d'hôpitaux et de machines variés.

Les cinq familles de cas (« **cohortes** ») :

Code	Signification
GLI	gliome de l'adulte (la plus fréquente)
SSA	patients d'Afrique sub-saharienne
MEN	méningiome
MET	métastases
PED	tumeurs pédiatriques (enfants)

La difficulté : si on entraîne le programme surtout sur des gliomes adultes (les plus nombreux), il risque d'être mauvais sur les enfants ou les métastases. **Le but de GoAT est justement de ne laisser aucune cohorte derrière.** Le critère qui compte n'est donc pas la moyenne, mais **la performance sur la pire cohorte.**

Trois **règles dures** du concours structurent tout le code :

1. **Aucune donnée extérieure.** On n'a le droit d'utiliser *que* les données fournies par GoAT. Utiliser d'autres images, ou même un modèle « pré-réglé » sur une édition précédente de BraTS, entraîne la **disqualification**. On part donc de zéro (« *from scratch* »).
2. **Soumission sous forme de conteneur fermé.** À la fin, on livre une « boîte » logicielle (un *conteneur Docker*) qui doit tourner **sans accès à Internet**, sur une carte graphique précise (24 Go de mémoire), et traiter **tous** les cas de test en **moins de 8 heures**.
3. **Géométrie préservée.** L'image colorisée rendue doit avoir exactement les mêmes dimensions et le même repère spatial que l'image d'entrée.

Ces trois règles expliquent pourquoi une grande partie du code n'est pas de l'« IA » à proprement parler, mais des **garde-fous** qui empêchent de se faire disqualifier.

2. Le vocabulaire indispensable (expliqué simplement)

Modalité. Une IRM n'est pas une seule image : on prend la même tête sous plusieurs réglages d'appareil, qui font ressortir des tissus différents. GoAT en fournit **quatre** par patient : **t1n**, **t1c** (avec produit de contraste), **t2f** (FLAIR), **t2w**. On les empile comme les couches R/V/B d'une photo couleur, sauf qu'ici il y a 4 couches au lieu de 3.

Label (étiquette). L'image « réponse » dessinée par des experts, où chaque voxel (= pixel 3D) porte un numéro : **0** = fond, **1** = nécrose (NCR), **2** = œdème (ED), **3** = tumeur active (ET). Le modèle apprend à reproduire ce coloriage.

Région. Les médecins ne notent pas chaque étiquette séparément mais trois **regroupements** cliniquement utiles : - **ET** = tumeur active = {3} - **TC** (tumor core, cœur tumoral) = {1, 3} - **WT** (whole tumor, tumeur entière) = {1, 2, 3}

nnU-Net v2. C'est le « moteur » de segmentation que le projet utilise. Pensez-y comme à un **logiciel de référence, déjà champion**, capable de se configurer tout seul à partir des données. On ne le

réécrit pas ; on le **pilote** et on lui ajoute nos spécificités. C'est un logiciel externe que le code appelle via des commandes.

Backbone (« épine dorsale »). Au cœur de nnU-Net se trouve un réseau de neurones. Sa structure interne s'appelle le backbone. Ici c'est **ResEnc-L** (« Residual Encoder, Large ») : une version costaute et éprouvée. Le backbone est la *forme du cerveau artificiel* ; il faut ensuite l'entraîner.

Entraîner / hyperparamètres. Entraîner = montrer des milliers d'exemples au réseau pour qu'il ajuste ses millions de réglages internes. Les **hyperparamètres** sont les quelques gros boutons qu'un humain règle *avant* l'entraînement (vitesse d'apprentissage, durée, taille des images traitées d'un coup...). Dans ce projet, ces boutons sont volontairement **laissés vides** (`# SPECIALIST:`) tant qu'un spécialiste ne les a pas fixés — pour éviter qu'on invente des valeurs au hasard.

Cross-validation / folds. Pour mesurer honnêtement la qualité sans tricher, on découpe les données en 5 paquets (« folds »). On entraîne 5 fois, à chaque fois en gardant un paquet de côté pour tester. C'est la **validation croisée à 5 folds**.

Leave-one-domain-out (LODO). Variante propre à GoAT : on entraîne sur 4 cohortes et on teste sur la 5e, *jamais vue*. C'est la mesure la plus directe de la généralisation (« sait-il faire sur un type de tumeur qu'il n'a jamais rencontré ? »).

Dice et HD95. Deux notes sur 1 (Dice) ou en millimètres (HD95) qui comparent le coloriage du modèle à celui des experts. **Dice** mesure le recouvrement (1 = parfait). **HD95** mesure l'erreur de contour la plus grande (en mm ; plus petit = mieux).

Sweep (« balayage »). Lancer automatiquement *beaucoup* d'entraînements en faisant varier un bouton, pour trouver le meilleur réglage. Avec un supercalculateur (HPC) on peut en lancer des dizaines en parallèle.

TTA, ensemble, post-traitement. Trois astuces de fin de chaîne pour gagner quelques points. **TTA** : on montre l'image aussi en miroir et on moyenne les réponses. **Ensemble** : on fait voter plusieurs modèles. **Post-traitement** : on nettoie le résultat (ex. effacer de minuscules taches isolées, souvent fausses).

SSL / semi-supervisé. Utiliser aussi les images **sans réponse** (non étiquetées). Le modèle se sert de ses propres prédictions confiantes comme fausses-réponses (« pseudo-labels ») pour continuer à apprendre. C'est la nouveauté du projet (Phase 4), désormais codée (voir la section dédiée plus bas).

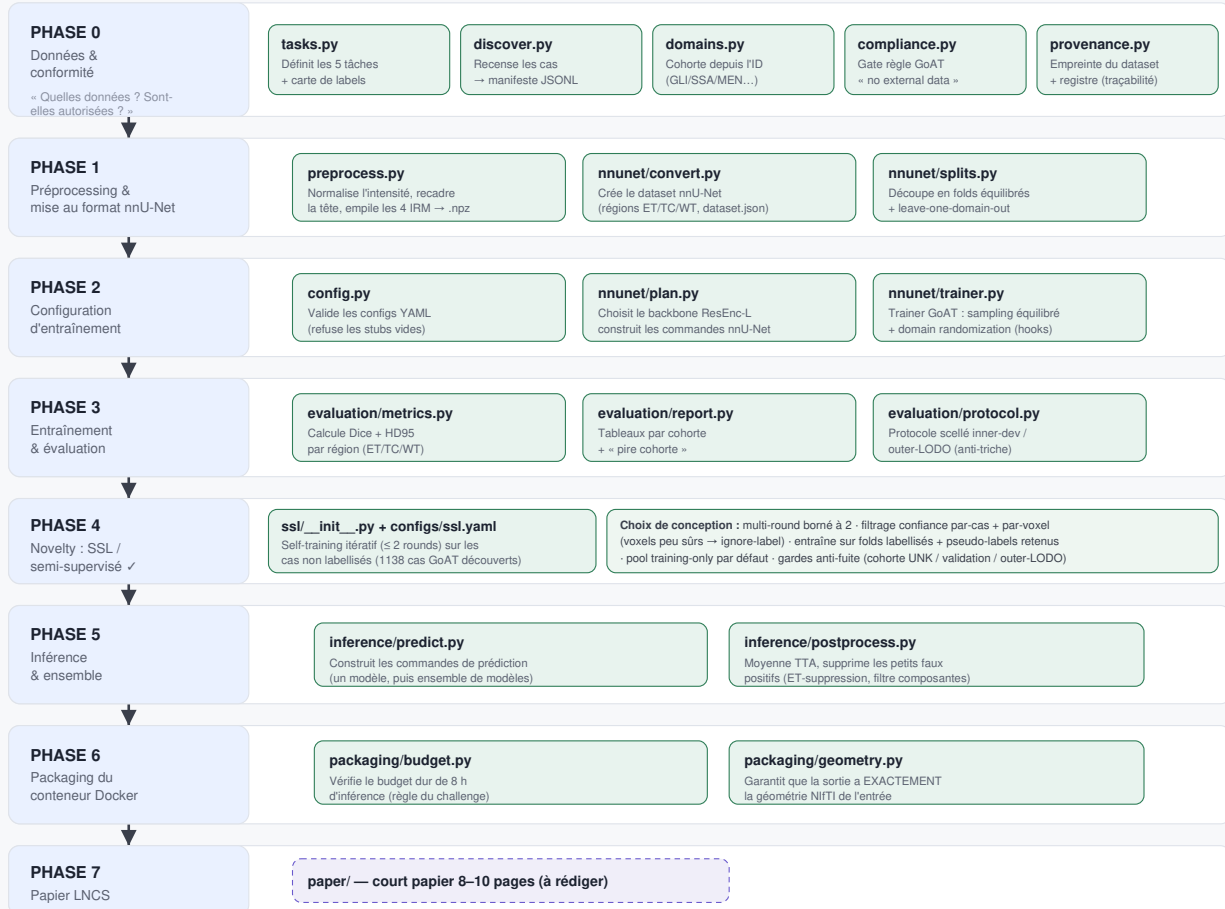
3. Vue d'ensemble : le schéma du pipeline

Le schéma ci-dessous présente l'enchaînement complet. Chaque rectangle vert est un fichier de code **réellement écrit et testé** ; les rectangles en pointillés sont encore à faire. Le flux se lit de haut en bas (Phase 0 → Phase 7).

BraTS 2026 — Task 3 (GoAT) · Pipeline du code

Un seul modèle de segmentation qui doit généraliser sur 5 cohortes (GLI · SSA · MEN · MET · PED). Chaque case = 4 IRM → 1 carte de tumeur.

`cli.py` — porte d'entrée en ligne de commande (commandes : `tasks` · `discover` · `preprocess-mri`). Orchestre les modules ci-dessous.



Légende : codé & testé (172 tests verts, dont 33 S) stub / à implémenter à rédiger ▲ Le code existe et est testé ; les hyperparamètres restent à régler.

(Le fichier `pipeline_brats_task3.svg` est dans le même dossier `docs/` . Le SVG s'affiche directement dans GitHub, VS Code et tout navigateur web.)

L'idée générale : on **part des données brutes** (en haut), on les **recense**, on les **nettoie**, on les **met au format** du moteur nnU-Net, on **entraîne et évalue**, on **améliore** (SSL), on **prédit et nettoie**, puis on **emballe** le tout dans le conteneur de soumission (en bas). En travers de tout cela, des **garde-fous** vérifient en permanence qu'on respecte les règles du concours.

4. Les 8 phases, fichier par fichier, fonction par fonction

Le code vit dans `src/brats2026/` . Voici chaque module, son rôle, et ce que fait chacune de ses fonctions — en langage simple.

Le chef d'orchestre : `cli.py`

C'est la **porte d'entrée** quand on tape une commande dans le terminal (`brats2026 ...`). Il ne calcule rien lui-même : il lit la commande et appelle le bon module.

Fonction	Ce qu'elle fait
<code>build_parser</code>	Décrit les commandes disponibles et leurs options.
<code>add_common_args</code>	Ajoute les options communes (<code>--data-root</code> , <code>--task</code>).
<code>cmd_tasks</code>	Commande <code>tasks</code> : affiche la liste des tâches connues.
<code>cmd_discover</code>	Commande <code>discover</code> : lance le recensement des cas.
<code>cmd_preprocess_mri</code>	Commande <code>preprocess-mri</code> : lance le nettoyage des IRM.
<code>main</code>	Point de départ : lit la ligne de commande et exécute.

À ce jour, seules **3 commandes** sont exposées (`tasks` , `discover` , `preprocess-mri`). Les commandes d'entraînement, d'inférence et d'emballage existent en code mais ne sont pas encore branchées sur le terminal.

PHASE 0 — Données & conformité

But : savoir *quelles* données on a, et vérifier qu'on a le *droit* de les utiliser.

`tasks.py` — la fiche d'identité de chaque tâche

Décrit, pour chacune des 5 tâches BraTS, où sont les données et comment lire les étiquettes. La Task 3 (GoAT) y déclare ses 4 modalités, sa carte de labels (`NCR=1`, `ED=2`, `ET=3`) et la règle « aucune donnée externe ».

Élément	Rôle
<code>TaskSpec</code>	Petite fiche structurée : nom, modalités d'entrée, étiquette cible, dossiers, carte de labels.
<code>TaskSpec.task_dir</code>	Renvoie le nom de dossier (<code>Task3</code>).
<code>TASKS</code>	Le dictionnaire de toutes les fiches (task1 → task5).
<code>get_task</code>	Retrouve une fiche par son nom (erreur claire si inconnu).
<code>task_base</code>	Calcule le chemin du dossier d'une tâche sur le disque.

discover.py — le recensement des cas

Parcourt le disque et dresse la **liste de tous les patients** disponibles, en notant pour chacun quels fichiers sont présents ou manquants. Produit un **manifeste** : un fichier texte avec une ligne par patient (format JSONL).

Fonction	Ce qu'elle fait
<code>CaseRecord</code>	Fiche d'un cas : identifiant, fichiers d'entrée, cible, manquants.
<code>iter_case_dirs</code>	Liste les sous-dossiers (un par patient).
<code>find_case_file</code>	Retrouve un fichier précis (ex. la modalité <code>t1c</code>).
<code>find_corrected_label</code>	Préfère une étiquette corrigée si elle existe (utile Task 1).
<code>discover_mri_cases</code>	Construit la liste complète des cas IRM.
<code>read_synapse_manifest</code>	Lit le fichier d'inventaire fourni avec les données.
<code>inspect_pathology_shard</code>	(Task 5 seulement) jette un œil dans une archive d'images.
<code>discover_pathology</code>	(Task 5) recense les données d'histologie.
<code>write_jsonl</code>	Écrit le manifeste, une ligne par cas.

domains.py — retrouver la cohorte de chaque patient

Astuce gratuite : la cohorte est écrite dans l'identifiant du patient (`BraTS-GLI-...` , `BraTS-PED-...`). Ce module la lit, ce qui permet ensuite d'équilibrer l'entraînement entre cohortes.

Fonction	Ce qu'elle fait
<code>COHORTS</code>	La liste officielle : GLI, SSA, MEN, MET, PED.
<code>cohort_from_case_id</code>	Extrait la cohorte d'un identifiant (renvoie <code>UNK</code> si inconnue).
<code>domain_table</code>	Associe chaque patient à sa cohorte.
<code>count_cohorts</code> / <code>CohortCounts</code>	Compte combien de cas par cohorte (et combien d'inconnus).

compliance.py — le gendarme « pas de triche »

Vérifie mécaniquement les règles dont la violation = disqualification. Chaque vérification renvoie `PASS` , `FAIL` ou `UNSURE` .

Fonction	Ce qu'elle vérifie
<code>check_data_provenance</code>	Tous les cas viennent bien des dossiers GoAT autorisés.
<code>check_no_prior_weights</code>	Aucun modèle « pré-réglé » suspect (mots-clés interdits : <code>pretrained</code> , <code>brats2023</code> ...).
<code>check_nas_not_committed</code>	Aucun chemin de données confidentielles n'a fuité dans le dépôt de code.
<code>gate_status</code>	Verdict global : un seul <code>FAIL</code> fait tout échouer.

`provenance.py` — le carnet de traçabilité (racine de confiance)

Prouve que l'entraînement est *reproductible* et *uniquement* à base de données GoAT. Il calcule une **empreinte** (un code unique, comme une empreinte digitale) du jeu de données — **sans jamais lire les pixels** (seulement la taille des fichiers et la géométrie d'en-tête, comme l'exige l'accord d'accès aux données).

Fonction	Ce qu'elle fait
<code>canonical_json</code> / <code>canonical_digest</code>	Transforment des informations en un code stable et reproductible.
<code>case_fingerprint</code>	Empreinte d'un seul cas (à partir de métadonnées, pas de pixels).
<code>dataset_fingerprint</code>	Empreinte de tout le jeu (insensible à l'ordre des cas).
<code>LedgerEntry</code> , <code>make_entry</code> , <code>append_entry</code> , <code>read_ledger</code>	Tiennent un registre : une ligne ajoutée par cas, jamais réécrite.
<code>fingerprint_from_ledger</code>	Recalcule l'empreinte à partir du seul registre (utile dans le conteneur hors-ligne).
<code>write_manifest</code>	Écrit un résumé lisible par un humain.
<code>nifti_header_fields</code>	Lit forme/affine d'un IRM sans charger les pixels .

Pourquoi tant d'efforts ? Pour pouvoir affirmer, preuve à l'appui : « ce modèle a été entraîné exactement sur ces données-là, et rien d'autre ». C'est l'assurance anti-disqualification du projet.

PHASE 1 — Préprocessing & mise au format nnU-Net

But : transformer les IRM brutes en données propres et standardisées.

`preprocess.py` — nettoyer et standardiser les IRM

Chaque appareil IRM produit des intensités différentes ; il faut les **harmoniser** pour que le modèle ne soit pas perturbé.

Fonction	Ce qu'elle fait
<code>require_mri_dependencies</code>	Vérifie que les bibliothèques d'image sont installées.
<code>robust_normalize</code>	Met toutes les images sur une échelle d'intensité comparable (en ignorant les valeurs extrêmes).
<code>foreground_bbox</code>	Recadre autour de la tête pour enlever le vide noir autour.
<code>load_manifest_records</code>	Relit le manifeste produit en Phase 0.
<code>preprocess_mri_manifest</code>	Orchestre tout : charge, recadre, normalise, empile les 4 modalités, enregistre en <code>.npz</code> .

`nnunet/convert.py` — parler la langue de nnU-Net

nnU-Net exige une organisation de fichiers très précise. Ce module la met en place, et déclare les **régions** ET/TC/WT.

Fonction	Ce qu'elle fait
<code>CHANNEL_ORDER</code>	Fixe l'ordre des 4 modalités (canaux 0→3).
<code>REGION_LABELS</code> / <code>REGIONS_CLASS_ORDER</code>	Déclarent les régions imbriquées (entraînement « region-based »).
<code>build_dataset_json</code>	Génère le fichier de description exigé par nnU-Net.
<code>LinkOp</code> / <code>ConversionPlan</code>	Décrivent à <i>l'avance</i> les copies de fichiers à faire (testable sans disque).
<code>plan_conversion</code>	Planifie : ne garde un cas que s'il a ses 4 modalités (+ étiquette).
<code>apply_conversion</code>	Exécute le plan via des liens symboliques (pour ne pas dupliquer les pixels et préserver la géométrie).

`nnunet/splits.py` — découper les données équitablement

Fonction	Ce qu'elle fait
<code>domain_balanced_kfold</code>	Crée 5 folds contenant chacun une part de chaque cohorte (aucun fold dominé par les gliomes).
<code>leave_one_domain_out</code>	Crée les découpes LODO (entraîner sur 4 cohortes, tester sur la 5e).
<code>write_splits</code>	Enregistre les découpes au format attendu par nnU-Net.

PHASE 2 — Configuration d'entraînement

But : décider *comment* on entraîne, sans inventer les chiffres au hasard.

`config.py` — le validateur de configuration

Les réglages d'entraînement vivent dans des fichiers YAML (`configs/train.yaml` , `configs/infer.yaml`). Ce module **refuse** une config tant qu'elle n'est pas complète : tous les boutons remplis **et** une « carte d'identité » (provenance) présente.

Fonction	Ce qu'elle fait
<code>make_template</code>	Crée un modèle de config volontairement invalide (tout à vide).
<code>validate_provenance_header</code>	Vérifie que la carte d'identité (commit, empreinte, graine aléatoire, date...) est remplie.
<code>validate_config</code> / <code>assert_valid_config</code>	Disent si la config est complète ; sinon nomment précisément ce qui manque.
<code>check_fingerprint_binding</code>	Vérifie que la config a bien été réglée sur les mêmes données que celles livrées.
<code>load_config</code> / <code>dump_config</code>	Lisent/écrivent les fichiers YAML.

À noter : les deux fichiers `configs/train.yaml` et `configs/infer.yaml` sont aujourd'hui des modèles vides (tout à `null`) ; le validateur les **rejette exprès** tant que le spécialiste ne les a pas remplis.

`nnunet/plan.py` — choisir le backbone et préparer les commandes

C'est ici qu'est **inscrit le choix du backbone** : **ResEnc-L**, configuration `3d_fullres` (traitement en 3D pleine résolution).

Fonction	Ce qu'elle fait
<code>RESENC_L_PLANNER</code> , <code>RESENC_L_PLANS</code> , <code>DEFAULT_CONFIG</code> , <code>GOAT_TRAINER</code>	Constantes : le backbone et le trainer choisis.
<code>nnunet_env</code>	Indique à nnU-Net où écrire ses fichiers (sous <code>work/</code> , jamais sur le NAS en lecture seule).
<code>plan_and_preprocess_command</code>	Fabrique la commande qui prépare les données pour ResEnc-L.
<code>train_command</code>	Fabrique la commande d'entraînement d' un fold.
<code>kfold_train_commands</code>	Fabrique les commandes des 5 folds (à lancer en parallèle sur le HPC).

Ce module **ne lance rien** : il *écrit les commandes*. C'est ce qui le rend testable sans carte graphique.

`nnunet/trainer.py` — le trainer « maison » GoAT

Définit `nnUNetTrainerGoAT`, une version personnalisée du trainer de nnU-Net qui ajoute les deux ingrédients clés de la généralisation : 1. **sampling équilibré par cohorte** (montrer autant chaque type de tumeur), 2. **domain randomization** (varier artificiellement contraste/bruit pour résister aux différences d'appareils).

Élément	Ce qu'il fait
<code>SPECIALIST_HOOKS</code>	La liste des 12 boutons que le spécialiste doit régler.
<code>GoATTrainerConfig</code>	La fiche de réglages ; chaque bouton vaut <code>None</code> (= « non décidé ») par défaut.
<code>unset_hooks</code>	Liste les boutons encore non réglés.
<code>assert_configured</code>	Refuse de lancer un entraînement tant qu'un bouton manque.
<code>domain_sampling_weights</code>	Calcule les poids de tirage par cohorte ; un curseur <code>alpha</code> va de « fréquence naturelle » (0) à « parfaitement égal » (1).
<code>nnUNetTrainerGoAT</code>	Le trainer concret (n'existe que si nnU-Net est installé).

Les 12 boutons (`initial_lr` , `num_epochs` , `batch_size` , `domain_sampling_alpha` , magnitudes de domain randomization, poids de la fonction de coût...) sont les **leviers de performance** à explorer par sweeps.

PHASE 3 — Entraînement & évaluation

But : mesurer la qualité, cohorte par cohorte, **sans se mentir**.

evaluation/metrics.py — calculer les notes

Fonction	Ce qu'elle fait
<code>validate_labels</code>	Vérifie que le coloriage ne contient que des étiquettes permises.
<code>region_mask</code>	Fabrique le masque d'une région (ET, TC ou WT).
<code>dice_coefficient</code>	La note de recouvrement (1 = parfait).
<code>hausdorff95</code> (+ <code>_surface_points</code> , <code>_directed_percentile</code>)	L'erreur de contour en mm (HD95).
<code>region_scores</code>	Calcule Dice + HD95 pour les 3 régions (mode « classique »).
<code>connected_components</code>	Repère les « taches » distinctes dans un masque.
<code>lesion_wise_dice</code>	Note par lésion : pénalise les fausses taches inventées (mode « lesion-wise »).

Le code calcule **deux familles de notes** (« classique » et « par lésion ») car on ne sait pas encore laquelle le portail officiel utilisera. On ne tranche pas à la place du concours.

evaluation/report.py — agréger en tableaux lisibles

Fonction	Ce qu'elle fait
<code>CaseScore</code>	Toutes les notes d'un cas, les deux familles côte à côte.
<code>hd95_summary</code>	Moyenne le HD95 en comptant à part les « ratés complets » (sinon un seul raté ruinerait la moyenne).
<code>per_cohort_table</code>	Tableau des notes par cohorte .
<code>worst_cohort</code>	Isole la pire cohorte — le chiffre qui compte pour GoAT.
<code>leave_one_domain_out_table</code>	Tableau LODO (une ligne par cohorte tenue à l'écart).
<code>write_report</code>	Enregistre le rapport en JSON.

evaluation/protocol.py — le protocole anti-triche scellé

C'est la pièce la plus astucieuse (détaillée en §5). Elle sépare *mécaniquement* deux choses : la zone où l'on a le droit de **régler** le modèle, et la mesure finale de généralisation qu'on n'a le droit de **regarder qu'une seule fois**.

Fonction	Ce qu'elle fait
<code>score_inner_dev</code>	La surface de réglage (5 folds), consultable autant qu'on veut.
<code>seal_config</code>	« Scelle » la config choisie en enregistrant son empreinte (écriture unique).
<code>reveal_outer_lodo</code>	Révèle la note finale LODO une seule fois , et seulement si la config n'a pas bougé.
<code>read_sealed_config</code>	Relit l'empreinte scellée.

PHASE 4 — Novelty : SSL / semi-supervisé ✓ (implémentée)

`ssl/__init__.py` + `configs/ssl.yaml`

L'idée en une phrase. Sur les données fournies, **1138 cas n'ont pas de réponse** (pas de coloriage de référence). Plutôt que de les jeter, on s'en sert : un premier modèle (le *teacher*, « professeur ») colorie ces cas, on **garde les coloriages les plus sûrs** comme s'ils étaient vrais (des « pseudo-réponses »), et on réentraîne un second modèle (le *student*, « élève ») sur l'ensemble. C'est l'**auto-apprentissage** (*self-training*), la contribution originale du projet.

Ce module ne lance pas le GPU lui-même : comme le reste du code, il **fabrique des commandes et une configuration** (les vrais réglages chiffrés vivent dans `configs/ssl.yaml`, propriété de l'« IA spécialiste »). Il est couvert par **33 tests**.

Les questions qui se sont posées, et les réponses retenues. Concevoir cette phase a demandé de trancher plusieurs choix ; ils ont été décidés par le « conseil » d'agents (spécialiste IA + gardien de conformité) puis validés :

- **Un seul passage, ou plusieurs ? → plusieurs, mais bornés à 2 rounds** (« multi-round »). Au 2^e tour, le modèle déjà amélioré recolorie les cas non étiquetés : utile surtout pour les cohortes les plus difficiles (SSA, PED, MET). Au-delà de 2, le modèle risque de **renforcer ses propres erreurs**. Le nombre de tours reste un bouton réglable (`n_rounds` , mettable à 1 si le temps de calcul manque).
- **Sur quelles données s'entraîne le modèle ?** → sur les **foldes (paquets) étiquetés habituels** *plus* les pseudo-réponses retenues, mélangés selon un **ratio étiquetés/pseudo** (réglable, ~2 pour 1) pour que les vraies réponses restent dominantes.
- **Comment décider qu'une pseudo-réponse est « assez sûre » ?** → un filtre à **deux niveaux** : *par voxel* (chaque point 3D peu sûr est marqué « à ignorer » et n'entre pas dans l'apprentissage) **et par cas** (un patient entier est rejeté s'il est trop incertain, presque vide, ou au contraire « halluciné »). On ajoute un **quota par cohorte** pour ne pas réinjecter surtout des gliomes adultes (majoritaires).
- **Quels cas non étiquetés a-t-on le droit d'utiliser ?** → **uniquement ceux du jeu d'entraînement** par défaut (bouton `allow_validation_pool` = faux). Le règlement n'autorise que les données fournies par GoAT ; le jeu de validation en fait partie, mais on l'exclut par prudence (éviter toute fuite vers le classement). Des **garde-fous automatiques** refusent tout cas

de cohorte inconnue (**UNK**), tout chevauchement avec le jeu de validation, et tout contact avec le pli « scellé » outer-LODO.

À retenir : aucune donnée extérieure, aucun modèle pré-entraîné d'un BraTS précédent — tout part de zéro, sur les seules données GoAT. C'est la règle dure du challenge, vérifiée par les garde-fous du module.

PHASE 5 — Inférence & ensemble

But : produire les coloriages finaux et les nettoyer.

`inference/predict.py` — fabriquer les commandes de prédiction

Fonction	Ce qu'elle fait
<code>predict_command</code>	Commande pour faire prédire un modèle (option pour couper le TTA si le temps presse).
<code>predict_ensemble_commands</code>	Une commande par membre de l' ensemble (chacun écrit dans son dossier).

`inference/postprocess.py` — nettoyer le résultat

Fonction	Ce qu'elle fait
<code>merge_tta</code>	Moyenne les prédictions des différentes vues (TTA miroir).
<code>connected_component_filter</code>	Supprime les taches plus petites qu'un seuil.
<code>et_suppression</code>	Reclasse les minuscules taches de tumeur active (souvent de faux positifs) en nécrose.

Le seuil de taille n'a **aucune valeur par défaut** : c'est un bouton spécialiste, fixé dans `configs/infer.yaml`.

PHASE 6 — Packaging du conteneur Docker

But : vérifier que la « boîte » de soumission respecte les règles dures.

packaging/budget.py — le chronomètre des 8 heures

Fonction	Ce qu'elle fait
<code>estimate_total_seconds</code>	Estime le temps total (temps par cas × nombre de cas).
<code>fits_budget</code> / <code>headroom_seconds</code>	Dit si ça rentre dans 8 h, et la marge restante.
<code>assert_within_budget</code>	Bloque si le temps projeté dépasse 8 h, en chiffrant le dépassement.

`BUDGET_HOURS = 8` est une **constante** (règle du concours), pas un bouton réglable.

packaging/geometry.py — le gardien de la géométrie

Fonction	Ce qu'elle fait
<code>geometry_matches</code>	Vérifie que la sortie a exactement la même forme et le même repère que l'entrée.
<code>assert_same_geometry</code>	Bloque et nomme précisément l'écart (forme ? repère ?) en cas de problème.
<code>read_geometry</code>	Lit forme/repère d'un IRM sans charger les pixels .

PHASE 7 — Papier LNCS

Le dossier `paper/` accueillera le court article scientifique obligatoire (8–10 pages) décrivant la méthode et les résultats par cohorte. **À rédiger** une fois de vrais résultats disponibles.

5. Les deux idées les plus astucieuses du code

A) L'empreinte du dataset (`provenance.py` + `config.py`)

Problème : comment prouver qu'on n'a pas triché et que tout est reproductible ? Solution : on calcule une **empreinte unique** du jeu de données (à partir de métadonnées seulement, jamais des pixels). Cette empreinte est : - **insensible à l'ordre** des cas (on les trie avant de calculer), - **sans horloge ni chemin machine** (donc recalculable à l'identique plus tard, même dans le conteneur hors-ligne).

Ensuite, `config.py` **exige** que chaque fichier de réglages porte cette empreinte (`check_fingerprint_binding`). Conséquence : impossible de régler le modèle sur un jeu de données et d'en livrer un autre — les empreintes ne correspondraient plus.

B) Le protocole scellé inner-dev / outer-LODO (`evaluation/protocol.py`)

Problème classique en IA : si on **regarde** la note finale puis qu'on retouche le modèle pour l'améliorer, on finit par « apprendre par cœur » le test et on se ment sur la vraie performance.

La parade, codée en dur : 1. **Zone de réglage libre** (`score_inner_dev` , sur 5 folds) : on peut la consulter autant qu'on veut. Elle ne contient **aucune** information sur la note finale. 2. **Note finale scellée** (`reveal_outer_lodo` , la vraie mesure de généralisation) : - on doit d'abord **figer** la config avec `seal_config` (qui enregistre son empreinte et **refuse d'être ré-écrit**) ; - la note finale ne peut être révélée **qu'une seule fois** (un fichier-drapeau `.outer_sealed` empêche tout second coup d'œil) ; - et seulement si la config n'a **pas changé** depuis le scellage.

Autrement dit : **on ne peut pas regarder deux fois, ni retoucher après avoir vu**. C'est une garantie d'honnêteté scientifique inscrite dans le code lui-même.

6. État actuel et ce qu'il reste à faire

Ce qui est fait (et couvert par 172 tests automatiques) : - toute la chaîne de **préparation des données** (Phases 0–1) ; - la **structure** d'entraînement, d'évaluation, d'inférence et d'emballage (Phases 2–3, 5–6) ; - la **Phase 4 (SSL / auto-apprentissage)**, désormais codée (+33 tests) avec sa configuration `configs/ssl.yaml` ; - tous les **garde-fous** (conformité, traçabilité, budget 8 h, géométrie, protocole anti-triche).

Ce qui manque : - **L'entraînement réel** : la machine est prête (GPU RTX 3090 24 Go, `torch` + `nnU-Net` installés, données converties en `Dataset501_BraTSGoAT`), mais aucun run n'a encore tourné — un **smoke-test 1 fold / ~5-10 epochs** est la prochaine étape. - **Le réglage des boutons** `# SPECIALIST:` d'entraînement (`configs/train.yaml`). - **Brancher** entraînement/inférence/emballage sur la ligne de commande. - **Finaliser** le conteneur Docker et **rédiger** le papier.

Deux constats récents importants : - **Métrique du classement** : le leaderboard officiel BraTS 2026 classe sur **Dice + NSD** (*Normalized Surface Distance*, tolérances 0,5 et 1,0 mm), **pas HD95**. Notre code calcule aujourd'hui HD95 ; il faudra **ajouter le NSD** pour coller au classement réel. -

Cohortes invisibles sur les données 2024 : les cas fournis ont des identifiants `BraTS-GoAT-NNNNN` qui **n'encodent pas la cohorte** (GLI/SSA/MEN/MET/PED). Tant que les données 2026 (préfixes `BraTS-<cohorte>-...`) ne sont pas là, le **rééquilibrage par cohorte** et le **leave-one-domain-out** ne peuvent pas s'appuyer sur l'identifiant.

Image mentale juste : l'usine est construite, inspectée et certifiée conforme ; la matière première est chargée et le moteur (GPU) tourne. Il reste à tourner les molettes (hyperparamètres) et à appuyer pour de bon sur le bouton « marche » (l'entraînement).

7. Mini-glossaire

- **Voxel** : un pixel en 3D.

- **Segmentation** : colorier automatiquement les zones d'intérêt (ici la tumeur).
 - **Modalité** : une variante de l'IRM (t1n, t1c, t2f, t2w).
 - **Cohorte / domaine** : une famille de cas (GLI, SSA, MEN, MET, PED).
 - **Backbone** : la structure interne du réseau de neurones (ici ResEnc-L).
 - **nnU-Net v2** : le moteur de segmentation de référence, piloté par ce projet.
 - **Hyperparamètre** : un gros bouton réglé avant l'entraînement.
 - **Fold** : un des paquets de la validation croisée.
 - **LODO** : entraîner sur 4 cohortes, tester sur la 5e jamais vue.
 - **Dice / HD95** : notes de recouvrement / d'erreur de contour.
 - **TTA / ensemble / post-traitement** : astuces de fin de chaîne pour gagner des points.
 - **SSL** : apprendre aussi à partir des images sans réponse.
 - **Sweep** : balayer automatiquement les réglages pour trouver le meilleur.
 - **Empreinte (fingerprint)** : code unique prouvant *quelles* données ont servi.
 - **Stub** : fichier-coquille, structure posée mais pas encore remplie.
-

Annexe — Les 12 hooks # SPECIALIST: d'entraînement

Ce sont les 12 « boutons » d'entraînement laissés **volontairement vides** dans le code (`GoATTrainerConfig` , fichier `nnunet/trainer.py`) et listés dans `SPECIALIST_HOOKS` . Tant que l'un d'eux vaut `None` , `assert_configured` **refuse de lancer** l'entraînement. La colonne « valeur par défaut » est la valeur de référence de nnU-Net **notée en commentaire** dans le code, **jamais appliquée en silence** : elle sert seulement de point de départ raisonnable pour les sweeps.

#	Hook	À quoi ça sert (en clair)	Défaut nnU-Net (référence)
1	<code>initial_lr</code>	Vitesse d'apprentissage initiale : à quelle vitesse le réseau ajuste ses réglages. Trop haut = instable ; trop bas = trop lent.	<code>1e-2</code> (0,01)
2	<code>weight_decay</code>	Frein anti-surapprentissage : empêche les réglages internes de devenir trop gros.	<code>3e-5</code>
3	<code>num_epochs</code>	Nombre de passages complets sur les données. Plus = mieux mais plus long (et borné par les 8 h).	<code>1000</code>
4	<code>batch_size</code>	Combien d'images traitées d'un coup. Limité par la mémoire GPU (carte A10G 24 Go).	déduit par le planificateur
5	<code>patch_size</code>	Taille du morceau d'image 3D traité à la fois (on ne traite pas tout le cerveau d'un coup).	déduit par le planificateur
6	<code>oversample_foreground_percent</code>	Part des morceaux qui doivent contenir de la tumeur (sinon le réseau ne verrait que du tissu sain).	<code>0.33</code> (33 %)
7	<code>loss_dice_weight</code>	Poids de la composante « recouvrement » dans la note d'erreur d'entraînement.	spécifique au projet
8	<code>loss_ce_weight</code>	Poids de la composante « classification par voxel » dans la note d'erreur.	spécifique au projet
9	<code>domain_sampling_alpha</code>	Levier #1 de généralisation. Curseur d'équilibre entre cohortes : <code>0</code> = on respecte les proportions réelles (beaucoup de GLI) ; <code>1</code> = on tire chaque cohorte à parts égales.	à fixer
10	<code>domain_rand_intensity_sigma</code>	Force du bruit ajouté sur l' intensité (simuler des appareils différents).	à fixer
11	<code>domain_rand_contrast_range</code>	Plage de variation du contraste (correction gamma) appliquée aléatoirement.	à fixer
12	<code>domain_rand_noise_sigma</code>	Force du bruit additif ajouté aux images (robustesse).	à fixer

Les hooks **1 à 8** sont les réglages d'entraînement « classiques » de tout réseau. Les hooks **9 à 12** sont la **spécificité GoAT** : ils servent directement la généralisation entre cohortes et entre appareils, et ce sont eux qui justifient les sweeps prioritaires sur le HPC (notamment le hook 9, `domain_sampling_alpha`, et le trio 10–12 de domain randomization).

Rappel : les 5 hooks d'inférence (`configs/infer.yaml`)

En plus des 12 hooks d'entraînement, 5 boutons concernent la **prédiction finale** (listés dans `INFER_HOOK_KEYS` , fichier `config.py`) :

Hook	À quoi ça sert
<code>tta_axes</code>	Les axes de mise en miroir pour le TTA (ex. <code>[0,1,2]</code> , ou <code>[]</code> pour désactiver et gagner du temps).
<code>ensemble_members</code>	La liste des modèles à faire voter ensemble.
<code>ensemble_weights</code>	Le poids de vote de chaque modèle.
<code>et_suppression_min_voxels</code>	Taille en dessous de laquelle une tache de tumeur active est effacée (faux positif).
<code>cc_min_voxels</code>	Taille minimale d'une tache pour être gardée (filtre de petites composantes).

Au total, **17 boutons** (12 + 5) sont à régler avant une vraie soumission. Aucun n'a de valeur appliquée par défaut : c'est un choix de conception pour qu'aucun chiffre ne soit « inventé » sans décision explicite.

Document généré à partir de la lecture intégrale des 24 modules de `src/brats2026/` . Pour aller plus loin : `CLAUDE.md` (guide projet), `ETAT_PROJET.md` (avancement), `PIPELINE_nnUNetv2_Task3.md` (stratégie détaillée).